Algorithmes distribués
ENSISA IR
TP 1

Sujet

On désire diffuser une information à tous les processus d'un système distribué dont le réseau de communication forme un graphe connexe quelconque.

Pour distribuer le contrôle, on utilise l'**algorithme de l'écho de Chang et Segall**.

L'initiateur de l'algorithme de l'écho envoie un message qui contient un petit texte. Chaque processus du système distribué affiche ce texte avec son identificateur sur la console lorsqu'il le reçoit la **première fois**.

Contexte

Vous utilisez un **simulateur** d'un système distribué qui permet la mise en œuvre de plusieurs topologies. Avec ce simulateur, les processus sont écrits en **langage Java**.

Un processus d'un système distribué est implémenté par une ***sous-classe*** de la classe ***Process*** du simulateur. La classe ***Process*** comporte des méthodes pour découvrir l'identité du processus, envoyer et recevoir des messages. Voir l'**API sur Moodle**. La classe ***Process*** et des classes supports sont encodées dans le fichier jar **processlibrary.jar**.

Pour coder le traitement qu'effectue un processus, il faut typiquement redéfinir la méthode ***run()*** de la classe ***Process***. Cette méthode est appelée par le simulateur lorsque la méthode statique ***main()*** appelle la méthode ***start()*** de ***Process***. Ainsi si ***MyProcess*** est le nom de la classe qui implémente un processus, le source sera typiquement :

```
public class MyProcess extends Process {
    public MyProcess() throws IOException {
    }

    public static void main(String[] args) throws Exception {
        var process = new MyProcess();
        process.start();
    }

    public void run() throws Exception {
        // Code source du processus, que ce soit un initiateur ou un suiveur
    }

}
```

Lorsque le simulateur est lancé, il instancie autant de processus UNIX que demandé, chaque processus exécutant une instance de *MyProcess*.

Vérifier le bon fonctionnement du simulateur

1. Téléchargez le dossier "**TP 1 - Fichiers supports**" depuis Moodle
2. Décompressez le fichier : **unzip Projet.zip**
3. **cd Projet**

   Dans ce dossier se trouve le programme de simulation (**simulator.jar**), la librairie cliente en bytecode Java (**processlibrary.jar**) et un dossier **ensisa** qui contient un exemple de processus écrit en java dans le paquetage *ensisa*.

4. Installez le SDK Java si ce n'est pas fait (version 21 minimum):

   sudo apt install jdk-...

5. Compilez les sources Java des processus

   javac -cp processlibrary.jar ensisa/TestMesh1.java

6. Lancez la simulation (4 est le nombre de processus (jusqu'à 20))

   java -jar simulator.jar -n 4 -mesh -classpath "processlibrary.jar:." ensisa.TestMesh1

   (Remplacez : par ; dans *classpath* sous Windows)

   Sur la console doit s'afficher ce texte (attention ce n'est pas déterministe) :

   Process 1 started
   Process 0 started
   Process 3 started
   Process 1 received message from 0 and replied

   Process 2 started
   Process 3 received message from 0 and replied

   Process 2 received message from 0 and replied

7. Regardez le source **TestMesh1.java** pour vous familiariser avec l'écriture d'un processus pour ce simulateur

   7.1. L'identité du processus figure dans la variable d'instance ***id*** (voir l'API du simulateur)

   7.2. ***outNeighbors*** contient la liste des identités des processus voisins de celui-ci auxquels il peut envoyer un message

7.3. ***inNeighbors*** contient la liste des identités des processus voisins de celui-ci desquels il peut recevoir des messages

7.4. La méthode d'envoi de message (***send***) envoie un message qui se présente sous la forme d'une instance de la classe ***OutputMessage***. ***OutputMessage*** contient un type de message et des données. Ces données se présentent sous la forme d'un flux de sortie Java (***DataOutputStream***) ; ce qui veut dire que vous pouvez utiliser des méthodes de ***DataOutputStream*** pour ajouter des données au message. ***OutputMessage*** fournit 2 méthodes utilitaires qui écrivent un entier ou une chaine de caractères dans le flux. Attention : ***DataOutputStream*** vous permet d'écrire des valeurs dans le message mais le message est envoyé en une seule fois sur le réseau par la méthode ***send***

7.5. Les méthodes de réception de message (***receive***, ...) retournent un objet instance de la classe ***InputMessage***. ***InputMessage*** contient l'identité du processus émetteur, un type de message et des données. Là encore, vous pouvez extraire les données en lisant un flux d'entrée Java (***DataInputStream***). Le flux ne permet de lire que les données du message, pas les données entrantes sur le réseau. ***InputMessage*** fournit 2 méthodes utilitaires pour directement extraire un entier ou une chaine de caractères du flux

7.6. Pour définir les types de message, vous pouvez déclarer un type énuméré comme dans l'exemple, mais ce n'est pas obligatoire.

Exercice

8. Ecrivez un source java pour implémenter l'**algorithme de l'écho de Chang et Segall** et diffuser un texte comme énoncé dans le sujet.

9. Générez le programme et exécutez-le dans le simulateur en faisant varier le nombre de processus et la topologie (*-bidirectional, -mesh ou -random*)

10. Modifiez le programme pour faire remonter une information vers l'initiateur.

    10.1. Par exemple, le message qui circule sur le réseau comporte non seulement le texte à afficher mais aussi un ensemble d'identificateurs de processus.

    10.2. Un processus suiveur doit mémoriser dans un ensemble local, tous les identificateurs de processus qu'il extrait des messages qu'il reçoit.

    10.3. Avant d'envoyer un message en retour à son père, chaque processus suiveur y fait figurer son numéro et les identificateurs mémorisés.

    10.4. Le processus initiateur affiche les ensembles d'identificateurs reçus.